



Collaboration optimization in software process composition



Andréa Magalhães Magdaleno^{a,*}, Marcio de Oliveira Barros^{b,2}, Cláudia Maria Lima Werner^{a,1},
Renata Mendes de Araujo^{b,2}, Carlos Freud Alves Batista^{c,3}

^a COPPE/UFRJ – Systems Engineering and Computer Science Program – Federal University of Rio de Janeiro (UFRJ), P.O. Box 68511, 21945-970 Rio de Janeiro – RJ, Brazil.

^b Post-graduate Information Systems Program, Federal University of the State of Rio de Janeiro (PPGI/UNIRIO), Av. Pasteur 458, 22290-240 Urca – Rio de Janeiro – RJ, Brazil.

^c Petrobras TIC/CPSW/PGOD/PMSW Rua da Assembleia, 100, 20011-904 Centro – Rio de Janeiro – RJ, Brazil.

ARTICLE INFO

Article history:

Received 30 November 2013

Revised 6 October 2014

Accepted 15 November 2014

Available online 3 December 2014

Keywords:

Collaboration

Software process

SBSE

ABSTRACT

Purpose: The purpose of this paper is to describe an optimization approach to maximize collaboration in software process composition. The research question is: how to compose a process for a specific software development project context aiming to maximize collaboration among team members? The optimization approach uses heuristic search algorithms to navigate the solution space and look for acceptable solutions.

Design/methodology/approach: The process composition approach was evaluated through an experimental study conducted in the context of a large oil company in Brazil. The objective was to evaluate the feasibility of composing processes for three software development projects. We have also compared genetic algorithm (GA) and hill climbing (HC) algorithms driving the optimization with a simple random search (RS) in order to determine which would be more effective in addressing the problem. In addition, human specialist point-of-view was explored to verify if the composed processes were in accordance with his/her expectations regarding size, complexity, diversity, and reasonable sequence of components.

Findings: The main findings indicate that GA is more effective (best results regarding the fitness function) than HC and RS in the search of solutions for collaboration optimization in software process composition for large instances. However, all algorithms are competitive for small instances and even brute force can be a feasible alternative in such a context. These SBSE results were complemented by the feedback given by specialist, indicating his satisfaction with the correctness, diversity, adherence to the project context, and support to the project manager during the decision making in process composition.

Research limitations: This work was evaluated in the context of a single company and used only three project instances. Due to confidentiality restrictions, the data describing these instances could not be disclosed to be used in other research works. The reduced size of the sample prevents generalization for other types of projects or different contexts.

Implications: This research is important for practitioners who are facing challenges to handle diversity in software process definition, since it proposes an approach based on context, reuse and process composition. It also contributes to research on collaboration by presenting a collaboration management solution (COMPOOTIM) that includes both an approach to introduce collaboration in organizations through software processes and a collaboration measurement strategy. From the standpoint of software engineering looking for collaborative solutions in distributed software development, free/open source software, agile, and ecosystems initiatives, the results also indicate how to increase collaboration in software development.

Originality/value: This work proposes a systematic strategy to manage collaboration in software development process composition. Moreover, it brings together a mix of computer-oriented and human-oriented studies on the search-based software engineering (SBSE) research area. Finally, this work expands the body of knowledge in SBSE to the field of software process which has not been properly explored by former research.

© 2014 Elsevier Inc. All rights reserved.

* Corresponding author. Tel.: +55 21 2562 8702; fax: +55 21 2562 8676.

E-mail addresses: andrea@cos.ufrj.br (A.M. Magdaleno), marcio.barros@uniriotec.br (M. de Oliveira Barros), werner@cos.ufrj.br (C.M.L. Werner), renata.araujo@uniriotec.br (R.M. de Araujo), carlos.freud@petrobras.com.br (C.F.A. Batista).

¹ Tel.: +55 21 2562 8702; fax: +55 21 2562 8676.

² Tel.: +55 21 2530 8088; fax: +55 21 2530 8051.

³ Tel.: +55 21 3487 5213; fax: +55 21 3487 5213.

1. Introduction

Software development is a complex process that involves the interaction of several people over a period of time to achieve a set of common goals. Whitehead et al. (2010) affirm that "collaboration is pervasive throughout software engineering", because almost all non-trivial software projects require effort and talent of many people working together. The team must communicate, share knowledge and artifacts, and coordinate their work. People play a major role in the success of software projects because they participate in the entire lifecycle acting in different roles (customers, users, managers, analysts, developers, etc.) (Altmann and Pomberger, 1999; DeMarco and Lister, 1999; Mistrik et al., 2010; Yildirim, 2006).

Regardless of all known benefits, achieving effective team collaboration remains a challenge. Despite recognizing that collaboration is advantageous, many organizations still do not know how to properly encourage it (Borrelli et al., 1995). Software engineering (SE) researchers still discuss which practices, processes, and tools are able to foster collaboration and to monitor its implementation (Araujo and Borges, 2007; Mistrik et al., 2010). Like other process management domains (Fischer, 2011; Schönthaler et al., 2012), collaboration is key for organizations to cope with the challenges of the modern business environment, which includes flexibility, adaptation, and open innovation (Chesbrough, 2003).

This fact is reinforced by research on distributed software development (DSD) (Cataldo and Herbsleb, 2008; Herbsleb et al., 2005), agile methods (Beck et al., 2001), free/open source software (FOSS) (Raymond, 2001), and ecosystems (Bosch and Bosch-Sijtsema, 2010). In these research areas, the focus on people (their talents, skills and knowledge) and concern with collaboration and communication appear recurrently.

On the other hand, collaboration should only be adopted when it has the potential to produce better results than individuals working alone (Hansen, 2009). In some cases, substantial time and resources are devoted to increase collaboration among team members, but end up being consumed without yielding the desired benefits. Therefore, it is important to determine when collaboration is truly needed and in what intensity (Hansen, 2009). We argue that collaboration can be systematically encouraged in software development organizations by explicitly considering it as part of organization processes.

A **software process** based approach can be useful to foster collaboration. The assumption that the adopted software process directly influences the quality of the developed product has motivated many organizations to invest in defining the processes which they use to produce software products (Cugola and Ghezzi, 1998; Pressman, 2001; Raman, 2000). A software process is "a coherent set of policies, organizational structures, technologies, procedures, and artifacts required to design, develop, deploy and maintain a software product" (Fuggetta, 2000). In general, a process is a way for the organization to plan the work and resources required to perform it, according to their goals. It defines how the organization works, how their activities should be performed, and the roles that people carry in its execution.

However, process definition initiatives are usually challenged by **diversity** (Lindvall and Rus, 2000; Siebel et al., 2003) in various levels. Different kinds of organizations, projects, development models, people and teams make it harder to define specialized processes to cope with known and new development contexts (Magdaleno et al., 2012). To cope with diversity involves the adaptation to the **context** of projects and teams and **reuse** of past experiences in the definition of software processes. **Process composition** is a way to promote the reuse of knowledge related to software processes (Magdaleno et al., 2012). When composing a process based on smaller and reusable units (such as process components or process lines), one can introduce collaboration as the central aspect of interest to be optimized for the combination of units that will form the process for a given project.

To manage collaboration while composing software processes, we created **COMPOOTIM** (Magdaleno et al., 2012). COMPOOTIM supports collaboration management in software processes. It comprises planning, composing, optimizing, and monitoring of software processes with the goal of maximizing collaboration among the members of a team assigned to develop a software project. Each of these four stages is designed to solve some of the challenges of managing collaboration in software processes and contains some proposed solutions.

This paper is dedicated to the composition and optimization stages and the alternatives investigated to find (close to) optimal combinations of process components that might yield a software process which maximizes collaboration among actors performing its activities. Optimization methods can provide solutions to complex problems and can suggest ways to find acceptable solutions in situations where perfect solutions are theoretically impossible or practically unfeasible (Harman and Jones, 2001). Search-based software engineering (SBSE) consists of a research area that studies software engineering from the perspective of optimization and search problems. It applies heuristic techniques to find approximate (near-optimal) solutions for complex problems whose optimal solution could not be found for all instances, because the search space grows exponentially with the size of those instances (Harman et al., 2009).

The optimization stage in COMPOOTIM includes: i) the formal modeling of collaboration in software processes as a SBSE problem (Harman and Jones, 2001); ii) the definition of a collaboration measurement strategy; and iii) the implementation of heuristic search algorithms to address the problem.

This optimization approach was evaluated in the context of a large oil company in Rio de Janeiro, Brazil. The main goal was to evaluate if COMPOOTIM was able to compose processes optimizing collaboration for three software development projects, according to their contexts. This experiment used random search (RS), hill climbing (HC) and genetic algorithms (GA) to find solutions for the process composition with collaboration optimization (PCCO) problem. Brute Force (BF) was also implemented to be used as a benchmark for comparing the algorithms results for small instances.

Algorithms were compared considering the resulting fitness. Results show evidence that GA is more effective (best fitness results) than HC and RS in the search of solutions for collaboration optimization in software process composition for large instances. However, all algorithms are competitive for small instances and even brute force can be a feasible alternative in such a context. These results were complemented by specialist feedback indicating satisfaction with correctness, diversity of options, adherence to the project context, and size and completeness in accordance with the expectations.

The remainder of this paper is structured as follows. In [Section 2](#), the background in collaboration, process reuse and composition is provided. [Section 3](#) summarizes the COMPOOTIM approach. The process composition with collaboration optimization problem is described in [Section 4](#). [Section 5](#) is dedicated to the design of the experimental study that evaluated the optimization of collaboration in the composition of software processes. In [Section 6](#), the results and threats to validity of the evaluation are analyzed. In [Section 7](#), some related works are discussed. Finally, [Section 8](#) concludes the paper and indicates some opportunities for future work.

2. Background

This section summarizes the main background topics of this work. The first one in [Section 2.1](#) is collaboration and its supporting aspects (communication, coordination, group memory and awareness). Then, in [Section 2.2](#), the software process reuse concept, benefits and approaches are presented. Finally, [Section 2.3](#) is dedicated to software process composition including process components and process lines.

2.1. Collaboration

Vreede and Briggs (2005) define collaboration as making a joint effort towards a goal. Because of its objectivity and simplicity, we adopted this definition for the present research. By collaborating, people combine their expertise, insights, and resources in order to perform a group activity, accomplishing more than they could as separate individuals (Dean et al., 2006; DeMarco and Lister, 1999).

The CSCW (*Computer Supported Cooperative Work*) research area boasts of extensive literature discussing the main aspects comprising group interaction and collaboration. The area usually adopts four group supporting aspects to characterize collaboration: communication, coordination, awareness, and group memory (Araujo and Borges, 2007; Ellis et al., 1991; Gerosa et al., 2006; Khoshafian and Buckiewicz, 1995; Mistrik et al., 2010). These group supporting aspects cannot be considered in isolation, because they are closely interrelated and dependent on each other. To collaborate, people have to exchange information (communication), organize the work (coordination), and operate together in a collective workspace (memory). Through awareness, the individual is informed about what is happening and get the necessary information to perform its role in the required activities.

Participants need to **communicate** in order to coordinate their work, assign tasks, make decisions, and solve problems. Within a group, communication can either be performed in real time (synchronous) or in different time (asynchronous). In synchronous mode, participants must be simultaneously available and the message sent by one is immediately received by the other(s), improving the speed of communication. In asynchronous mode, despite the potential loss in speed, messages sent by one participant are received in a future and unknown moment by its target audience, allowing them as much time as needed to think and provide a proper answer before continuing the communication process.

Coordination is the aspect that organizes the group to guarantee that tasks are performed as expected, i.e., within an expected period of time, following a desired sequence and respecting established constraints. Coordination should avoid conflicts, while allowing the group to be productive. It comprises the pre-articulation of tasks, their follow-up, and the post-articulation of group activities. Coordination means 'to keep the group alive', stimulating contributions and establishing dynamics.

Group memory records information related to the execution of the group activity. It preserves both the formal knowledge obtained through the interaction – documents and artifacts produced by executing the required activities – and informal knowledge – decisions, ideas, comments, perceptions, etc. – the rationale through which documents and artifacts had been created.

Finally, **awareness** relates to the understanding of others' activities in order to contextualize individual contributions (Dourish and Bellotti, 1992). Participants should clearly understand the group's common objective, the role of each member, what must be done, and the results and impacts of each member's activities.

The main advantages of collaboration can be summarized as follows (Alonso et al., 2010; DeMarco and Lister, 1999): reduction of time required to perform tasks; improvement of the ability to solve complex problems; increase of the ability to generate creative alternatives; discussion of the advantages and disadvantages of each alternative to select the best course of action and to make rational decisions; communication improvement; learning; personal satisfaction; and encouraging innovation.

However, collaboration also faces some challenges, such as: delays in carrying out tasks collaboratively; lack of work coordination; high cost of decision making; and resistance to knowledge sharing. In these cases, substantial time and resources may be consumed in inefficient collaborative activities without yielding the desired benefits.

Therefore, the recognized challenge here is how to manage collaboration in software processes, maximizing the positive results of the

extra effort required to introduce its aspects in software processes. To satisfy this goal, the COMPOOTIM approach includes stages to plan, compose, optimize, and monitor collaboration in software processes, as detailed in Section 3.

2.2. Software process reuse

Software processes for specific projects may be defined in many ways, including process reuse. The reuse of software processes is "the ability to build new processes from existing processes", since different projects are likely to have common activities (Feiler and Humphrey, 1992). Relying on the use of previously built process models as parts in the construction of a new one, rather than building it from scratch, can result in a more efficient construction of the process model (Ribó and Franch, 2002).

Process reuse is intended to support the definition of processes based on successful past experiences. This opportunity to reuse the experience gained during the planning and conduction of previous projects enables the determination of the best strategies for future projects. Thus, reusing software processes is important to: disseminate the experiences of previous projects; reduce development costs and time (Hollenbach and Frakes, 1996); increase the quality and adequacy of the generated processes; promote the reuse of expert knowledge; and make process definition accessible to less experienced people. Xu and Ramesh (2008) add that modeling a process from the beginning, without taking advantage of previously existing processes, is a risky and laborious activity.

In practice, process reuse has two different approaches (Chrissis et al., 2006): tailoring a software project's process from the organizational standard process (Pedreira et al., 2007); or process composition, which synthesizes the process to be used to conduct a given project based on smaller and reusable units, known as process components. This work is based on the second strategy (process composition), which has the potential to deal with process diversity or variability, help to disseminate knowledge and successful experiences, and adapt software processes for utilization in a different context than the one to which it was originally defined.

2.3. Process composition

Despite being one of the main tasks to be executed by the project manager, process composition is not a simple task (Ternite, 2009). Such activity demands experience and knowledge of several aspects related to software engineering. It also requires pondering many organizational factors (requirements, needs, and goals) and evaluating a large set of constraints regarding project and team context. Understanding these factors and how they may affect the software process is a challenge to project managers (Xu and Ramesh, 2008), especially less experienced ones. The manager is not usually able to evaluate all available combinations and often chooses a process in an *ad-hoc* manner, possibly selecting one that is not the best alternative for the current project. These decisions are usually taken once, early in the project life-cycle, when there is not enough information to predict an appropriate process. Besides, project context will possibly change during its execution and the process should reflect it. In fact, the quantity and diversity of information involved make the process composition a complex, time-consuming and error-prone activity when performed without proper computer support.

As project budget, development time, and product quality directly depend on the quality of the software process, process composition needs to be correctly conducted in order to prevent the following negative consequences (Pedreira et al., 2007): unnecessary activities that lead to a waste of time and money; the omission of necessary activities, which may affect the product quality; and the failure to comply with the organizational or international standards.

Table 1
Process component structure and example.

PC – Daily meeting	
Role:	Project team
Optionality:	Mandatory
Required interfaces:	Product backlog
Provided interfaces:	Sprint burndown

We claim that collaboration is an aspect that can be planned and monitored in processes composed for different software development projects. Thus, we created the COMPOOTIM strategy for software process composition based on reusable units, such as process components (OMG, 2008; Park et al., 2006) and software process lines (SPrL) (Aleixo et al., 2010; Barreto et al., 2011; Rombach, 2006; Washizaki, 2006).

Process components are reusable items that can be connected to create new and more complex processes (Barreto et al., 2011; Magdaleno et al., 2007). There are several definitions of process components, but for the sake of this research a process component can be seen as an encapsulation of data and process behavior at a given level of granularity (Gary and Lindquist, 1999). A component is considered something relevant to be reused in other process definitions and encapsulates a number of process information, such as activities, required and produced work products, actors, among others. A process component has a *required interface*, which lists the artifacts the component uses to perform its job, and a *provided interface*, which lists the artifacts produced by the component after its execution. An example of process component is provided in Table 1.

Although the use of components for process composition provides many advantages, starting from small units for composing large processes seems to be insufficient, due to the fine granularity. As pointed out by experiments in the software reuse area, the use of reusable larger structures, such as software process lines, enhances the process composition (Barreto et al., 2011).

Software process line (SPrL) appears as a systematic technique for processes reuse, which applies the principles of software product lines in the realm of software processes. It allows creating a generic (set of) process(es) that capture(s) the commonalities and controlled variabilities across a domain (Rombach, 2006). SPrL can be understood as a set of processes in a particular domain or for a particular purpose, having common characteristics and built upon common, reusable process assets (Washizaki, 2006). SPrL prepares the processes of an organization to suit future anticipated (process) needs by determining a common process core and variable process parts that satisfy specific needs (Armbrust et al., 2008). The processes are designed from artifacts and can be reused and combined among themselves, according to composition rules. These composition rules can be inclusive or exclusive. They specify components that must be included or excluded depending on the presence of other components.

When composing a software process based on smaller and reusable units, one may decide which of the potential various combinations that can be built from a set of components is the best choice for a given software project. Therefore, a pair-wise comparison is required to determine which between two processes is the best for the current context. If the company has a large database of process components, the number of potential combinations grows exponentially. This presents a scenario in which a software engineering problem (software process composition) must be faced as an optimization problem whose space of potential solutions can be extensive. To allow finding close to optimal solutions to this problem without examining all potential alternatives, the process composition problem must be described as an optimization problem and heuristic search algorithms may be used to navigate its solution space looking for acceptable solutions according to the project under interest. To drive this search,

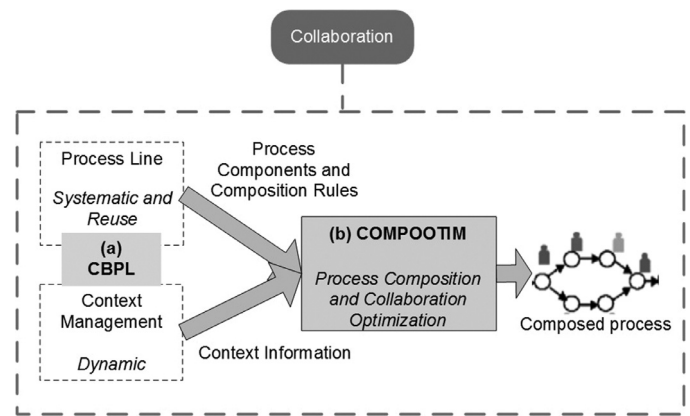


Fig. 1. COMPOOTIM approach.

this work suggests using collaboration as the aspect of interest to be optimized for this combination of units that will form the process.

3. COMPOOTIM

The COMPOOTIM⁴ (Magdaleno et al., 2012) approach aims to provide systematic guidance to plan, compose, optimize, and monitor collaboration in software processes. COMPOOTIM (Fig. 1) supports project managers' decisions about process components selection and combination and optimizes the collaboration while suggesting processes. The benefits are threefold: (a) automating some of the steps reduces the effort and increases productivity in process definition; (b) at the same time, automation can contribute to improve the quality and appropriateness of the resulting process; and, finally, (c) it adapts software processes to the context of organizations, projects, and teams and allows the reuse of past and successful experiences. We should emphasize that COMPOOTIM approach is not a replacement of a human manager decision. Instead, it supports project managers' decisions in planning a software process.

From the combination of SPrL and context management, a **context-based process line (CBPL)** (Nunes et al., 2010) offers systematic support to process composition and reuse. CBPL corresponds to a set of process components, organized to represent common and variant parts of a software process within a specific domain, that can be reused and combined (according to composition rules) to dynamically create and adapt processes. CBPL aims to establish relationships between a given situation and the components of the SPrL; understand the current context of a project through its context information (some examples of context information are team size, members' geographical location, and project duration); identify changes in organization, project, and team context throughout the project; and dynamically adapt the process to the new context (Nunes et al., 2011).

CBPL is responsible for the creation of an organizational process line. When organizational needs or goals change, CBPL is also responsible for promoting the evolution of the SPrL. The SPrL captures the commonalities and variabilities in a domain through a features model (Lee et al., 2002). It makes explicit the points where software processes are similar and can be reused, and the points where they diverge and need specific treatment. Variability in software processes becomes important in order to deal with diversity and adapt for a particular context. After establishing the SPrL, instead of defining a process from scratch for each project, project managers can make use of the SPrL infrastructure to compose a project-specific process. When process composition is finished, the process is ready to run.

⁴ CoSite: <http://reuse.cos.ufrj.br/cdsoft/compootim.en.html>.

COMPOOTIM uses the organization's SPRL as input. It receives process components, composition rules (represent dependencies or mutually exclusive relationships), and context information from CBPL (Fig. 1a), selects the most appropriate components for the context, combines these components in search for a feasible software process, and evaluates the effectiveness of this process. In the end, it suggests a set of alternative processes that maximize collaboration for the specific project context. Further details about COMPOOTIM's composition and optimization mechanisms are provided in Section 4.

In CBPL, the process is composed by the project manager from the process line. Thus, the manager makes decisions associated to variation points – points in the process where one can decide what variants, among the available ones, will be chosen to be executed. However, product line experience shows that this configuration can be time-consuming and expensive (Deelstra et al., 2005). The same difficulties are expected for process composition. Aiming to reduce the complexity of this activity, COMPOOTIM automates the composition of a process in accordance to the project's current context, infers and suggests which process components are adequate.

The project manager receives additional information about context and constraints that influence each of the options. COMPOOTIM provides the manager with a basis for systematic decision making, since it offers information that otherwise would be difficult to obtain and allows the manager to analyze different solutions and consider the impact of each on the project. Thus, it contributes to reduce the effort spent by the manager to configure the most appropriate software process to his/her project. Despite the automation support, the project manager has the final decision about the process that will actually be adopted.

The COMPOOTIM architecture consists of three main interconnected parts. The first one is the *composition mechanism*, implemented in Java using MVC architecture in a standalone solution. It is responsible for the SPRL creation and process components selection according to the project context. Context and composition rules act as filters, discarding components that do not apply to the project context and selecting those compatible with the context of the project.

Regarding optimization, process composition is understood as an SBSE problem due to the need to maximize collaboration for a given project context. Using heuristic algorithms, one can search for solutions that maximize the level of collaboration in the process composed for the project without traversing all alternative combinations. Therefore, the *optimization mechanism*⁵ is the second part of COMPOOTIM's architecture and is responsible for sequencing process components as well as selecting the best sequence on regards to collaboration. Sequencing connects the formerly selected components, according to their required interfaces (input artifacts) and provided interfaces (output artifacts), resulting in one or more possible processes to be used in the project. Then, it searches a solution space with several candidate processes, trying to find the set of components for which the fitness function has a maximum value. Its implementation uses the JMetal framework (Durillo et al., 2006).

Finally, the *visualization mechanism* is the presentation layer. It summarizes information about project and team context that influenced the suggestions of processes. It also describes the suggested processes by presenting their sequences of components and produced artifacts. In addition, it provides indicators to monitor the value of the process collaboration that was maximized.

This work focuses on the process composition and optimization stages of COMPOOTIM, since they are strongly connected with each other and both focus on the specific problem of finding suitable combinations of process components that maximize collaboration in the software process. Given a large number of components, the number

of process combinations may become huge and, thus, heuristic algorithms are used to explore the alternatives.

4. Process composition with collaboration optimization problem

Harman (2007) observes that two key ingredients must be available to allow the application of optimization in software engineering problems: a representation of the problem under interest and one or more fitness functions to evaluate the proposed solutions. Given these ingredients, one can apply optimization techniques to many SE areas, such as requirements engineering (Bagnall et al., 2001; Zhang et al., 2007), software design (Barros, 2012; Praditwong et al., 2011), project planning and cost estimation (Alba and Chicano, 2007; Antoniol et al., 2004; Barreto et al., 2008), testing (Li et al., 2007; Yoo and Harman, 2012), and quality assessment (Khoshgoftaar et al., 2004). Beyond these traditionally exploited research areas, it is also possible to expand SBSE to other areas such as business process management (BPM) aiming to discover actual process implementation in comparison with designed process models (Marchetto et al., 2011; Tomasi et al., 2012). Following this idea, we claim that this is also possible to apply the SBSE perspective in software process optimization and get interesting and potentially important results.

This section presents the definition of the PCCO problem (process composition with collaboration optimization), which represents the optimization problem underlying COMPOOTIM. It can be summarized by the following question: how to compose a process for a specific software development project context aiming to maximize collaboration among team members assigned to its execution? An initial model for this problem has been described in (Magdaleno, 2010a), but since then it has evolved. Nevertheless, having a clear model for the problem is critical to understand its nature and to identify proper SBSE techniques that are able to find its solutions.

4.1. Problem definition

An overview of the steps comprising the PCCO problem is presented in Fig. 2. The first stage involves the creation of combinations of process components that satisfy results expected from the SPRL (Fig. 2a), process components constraints, and restrictions related to the project context. Then, the collaboration induced by the process is calculated considering the collaboration practices that are part of each component. Finally, a mono-objective systematic search algorithm uses collaboration as fitness function to select the best software process, that is, the feasible combination of process components that maximizes collaboration. Feasibility evaluation depends on connecting the selected components among each other, respecting the artifacts provided or required by their interfaces to define the process sequence (Fig. 2b).

The PCCO problem receives three sets as inputs: (i) a set of process components retrieved from the organization's process knowledge base; (ii) a set of entry artifacts containing information required to start the software development project; and (iii) a set of resulting artifacts describing the expected results from executing the software process. It aims to produce a software process that maximizes collaboration among team members. This process is represented as a partial order of a subset of the process components received as input. The process must be feasible in the sense that all artifacts comprising the required interface of a given component C must be either in the set of entry artifacts or part of the provided interface (output artifacts) of another component appearing before C in the partial order.

Let PC be the set of C_i process components retrieved from the organization's SPRL and received as input by the PCCO. Finally, process components might be mandatory or optional, but this aspect is only partially related to the optimization – mandatory components are not subject to variation and selection by the optimization algorithm, but

⁵ Download at: <http://reuse.cos.ufrj.br/cdsoft/download.en.html>.

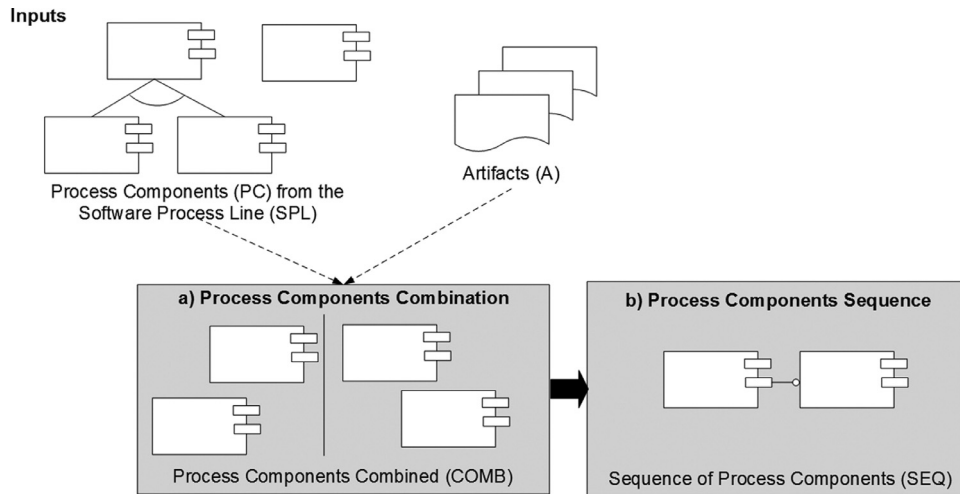


Fig. 2. PCCO optimization problem overview.

after that they are included in the final software process respecting the required and provided interfaces.

Let A be the set of artifacts involved in a software process. This set must contain all artifacts provided as input to the process, and all intermediate artifacts produced, transformed, or consumed as the process is executed. Let EA be the set of entry artifacts received as input by the PCCO and RA be the set of resulting artifacts also received as an input by the PCCO, RI_i be the set of artifacts comprising the required interface of component C_i , and PI_i be the set of artifacts comprising the produced interface of C_i . Considering the statements above, we can derive that:

$$EA \supseteq A$$

$$RA \supseteq A$$

$$RI_i \supseteq A, \quad \forall C_i \in PC$$

$$PI_i \supseteq A, \quad \forall C_i \in PC$$

First, we are interested in determining the set of feasible unordered software processes (combinations) that can be built from the available set of components (Fig. 2a). Each combination is a subset of the power set of PC , so that all entry artifacts required by any component involved is either provided as an input to the process (pertains to EA) or is produced by another component comprising the selection. Moreover, the candidate combination must be so that all expected results (artifacts pertaining to RA) must also be provided as an input to the process (EA) or be produced by another component in the combination. Formally, the set of all potential processes is represented by $COMB$ in the equation below.

$$COMB = \{s \subset 2^{PC} \mid (\forall C_i \in s \Rightarrow \forall a \in RI_i, \rightarrow a \in EA) \vee (\exists C_j \in s \wedge a \in PI_j) \wedge \forall a \in RA \rightarrow a \in EA \vee (\exists C_j \in s \wedge a \in PI_j)\}$$

The sequencing of process components (Fig. 2b) is based on their provided and required interfaces. The sequencing algorithm is presented in Table 2. It receives an unordered feasible set of components (an element of the $COMB$ set) along with the EA and RA sets as input and generates a partial order of components. The algorithm implements a topological sort in which components are added to an ordered-list as their required interfaces are supplied by components that were formerly added to the list. The topological ordering may fail if a circular dependency among components is found (for instance, if

a given component A requires an artifact produced by component B , which on its turn requires some artifact to be produced by A). Such would lead to an unfeasible process, which is rejected by the algorithm.

The algorithm starts by creating a placeholder component to represent the start of the process (lines 01–03). Following, a second placeholder is created to represent the end of the process and the production of the required artifacts (lines 05–07). Then, the list containing the partial order (PO) is initialized with the source placeholder (line 09), the set of produced artifacts (LPA) is initialized with the entry set of artifacts (line 10), and all components that need to be evaluated are placed in a list (line 11) which is processed until it is empty (line 13) or a circular dependency is found (line 25).

Lines 13–28 represent the main loop of the algorithm. In this loop, for each component C whose required interface can be satisfied by artifacts produced by components formerly added to the partial order (that is, components that already are in PO), component C is removed from CTE (line 18), added to PO (line 19), and LPA is updated to consider its produced interface (line 20). If no component is added to PO in a given step of the main loop, the condition in line 25 is triggered to signal an unfeasible selection of process components.

For performance considerations, the combination and sequencing phases are implemented in a single algorithm, which takes EA , RA , and a candidate solution as inputs and produces a partial order to be subjected to fitness evaluation. A candidate solution is a bit-vector of N entries, where entry n contains one if component n is considered in the combination, or zero, otherwise.

4.2. Fitness function

Fitness functions characterize what are considered good solutions and help to better understand the nature of candidate solutions. As our research problem involves optimizing collaboration in software processes, collaboration is used as a mono-objective fitness function. Thus, invalid combinations of components, which do not satisfy all expected artifacts (required interfaces), are penalized with fitness equivalent to zero. For those component combinations satisfying the requirements imposed by the SPL, a fitness value of β is calculated according to the collaboration measurement strategy detailed below.

In this work, the collaboration maturity model (CollabMM) (Magdaleno et al., 2008,2009) acts as a framework to define collaboration maturity levels (ad-hoc, planned, aware, and reflexive). The model summarizes the main characteristics and collaboration practices. CollabMM describes an evolutionary path in which processes can progressively achieve higher capability on collaboration.

Table 2
Sequencing algorithm.

Inputs:

- A set of components ($s \in \text{COMB}$).
- A set of entry artifacts available to start the software process (EA).
- A set of artifacts expected at the end of the process (RA).

Outputs:

- Partial order of process components (PO).

Algorithm:

```

01 Create a placeholder component to represent the start of the process: SOURCE
02 Let  $RI_{\text{SOURCE}} = \emptyset$ 
03 Let  $PI_{\text{SOURCE}} = \text{EA}$ 
04
05 Create a placeholder component to represent the end of the process: TARGET
06 Let  $RI_{\text{TARGET}} = \text{RA}$ 
07 Let  $PI_{\text{TARGET}} = \emptyset$ 
08
09  $PO \leftarrow \text{SOURCE}$ 
10  $LPA \leftarrow \text{EA}$ 
11  $CTE \leftarrow s \cup \text{TARGET}$ 
12
13 WHILE  $CTE \neq \emptyset$ 
14   Let Changed = false
15
16   FOR EACH C in CTE
17     IF  $RI_C \subseteq LPA$  THEN
18        $CTE \leftarrow CTE - C$ 
19        $PO \leftarrow PO \cup C$ 
20        $LPA \leftarrow LPA \cup PI_C$ 
21       Let Changed = true
22     END IF
23   END FOR
24
25   IF NOT Changed THEN
26     FAIL "No sequence can be built from the selected components"
27   END IF
28 END WHILE
29
30 RETURN PO

```

Table 3
Group supporting aspects and collaboration practices.

Group supporting aspects	Collaboration practices		
	Level 2	Level 3	Level 4
Communication	Communication planning (PR1)	Distribution of information (PR5)	Closure (PR9)
Coordination	Group work planning (PR2)	Work monitoring (PR6)	Evaluation (PR10)
Group memory	Integration of individual products (PR3)	Explicit knowledge sharing (PR7)	Tacit knowledge sharing (PR11)
Awareness	Social awareness (PR4)	Process awareness (PR8)	Collaboration awareness (PR12)

The framework supports organizations in explicitly incorporating collaboration practices into their processes, allowing this aspect to be systematically enforced during process modeling.

During the creation of a CBPL, software engineers have to evaluate each process component and determine whether it contributes to some of the 12 collaboration practices (PR) defined in CollabMM (listed in Table 3). Thus, each component contributes with a set of collaboration practices that improves the whole process collaboration level.

Taking the relationship between process components and CollabMM's practices into consideration, we calculate the level of collaboration for the whole process given the combination of process components that represents this process. We assume that the level of collaboration of the process depends on the collaboration practices that will be performed as part of the process according to the selected process components and their associated practices. Therefore, a map of all the collaboration practices present in the process is created from the union of all process components' collaboration practices.

The collaboration potential (CP) for a given process is calculated by counting the collaboration practices for each CollabMM level provided by the components comprising the process. The count of level 2 practices is multiplied by 2, representing the relative impor-

tance of these practices to their level 3 and level 4 counterparts, whose counts are, respectively, multiplied by 3 and 4. Results from these multiplications are then summed to result in the process' collaboration potential.

For instance, consider the three process components (PC1, PC2, and PC3) presented in Table 4 which shows the collaboration practices (PR) addressed by each component. Consider two possible software processes composed by subsets of these components, as shown in Table 5. Process X uses PC1 and PC2. Therefore, its map of collaboration practices includes PR1 to PR5 (being PR2 twice) and PR11. On the other hand, process Y is comprised of PC2 and PC3 and, therefore, contains eight collaboration practices (being PR2 twice). The potential collaboration calculated for processes X and Y is presented in Table 6, on which one can notice that process Y has higher collaboration potential than process X.

The size of the search space for the PCCO problem is 2^p , where p is the number of optional process components available in the organization's repository. A large repository cannot be searched in reasonable time by using exact methods. Thus, a solution using heuristic search methods is required to solve large instances of the PCCO problem. To settle this, we have evaluated some heuristic search algorithms to be implemented in COMPOOTIM, as described in the next section.

Table 4
Process components (PC) × collaboration practices (PR).

Process component	Collaboration practices											
	PR1	PR2	PR3	PR4	PR5	PR6	PR7	PR8	PR9	PR10	PR11	PR12
PC1	x	x	x	x								
PC2		x			x						x	
PC3		x	x				x		x	x		x

Table 5
Map of processes' collaboration practices.

Processes	Process components			Collaboration practices											
	PC1	PC2	PC3	PR1	PR2	PR3	PR4	PR5	PR6	PR7	PR8	PR9	PR10	PR11	PR12
Process X	x	x		x	x	x	x	x						x	
Process Y		x	x		x	x		x		x		x	x	x	x

Table 6
Final process collaboration.

Weight	Collaboration practices												Calculation	
	PR1 2	PR2 2	PR3 2	PR4 2	PR5 3	PR6 3	PR7 3	PR8 3	PR9 4	PR10 4	PR11 4	PR12 4	Collaboration	
Process X	X	x	x	x	x						x		17	
Process Y		x	x		x		x		x	x	x	x	28	

Table 7
Process component (project responsible assignment) example.

PC – register lessons learned	
Role:	Project manager
Optionality:	Optional
Required interfaces:	Project budget
	Project management plan
Provided interfaces:	Registration of lessons learned
Collaboration practices (PR):	PR7, PR8, PR9, PR10, PR11, and PR12
Collaboration potential (CP):	22

Table 7 shows an example of a real process component – register lessons learned – extracted from the organizational process used in our study. This process component was considered to contribute with PR7, PR8, PR9, PR10, PR11, and PR12 collaboration practices of Col-labMM. Therefore, it achieved a CP equals to 22. This information will be used by COMPOOTIM during software processes composition.

5. Evaluation planning

This section describes the design of the experimental study that evaluated the optimization of collaboration in the composition of software processes. This section encompasses the selected algorithms, the configurations under which they were executed, selected instances, and research questions.

5.1. Context

This study was conducted in the context of a large oil company in Rio de Janeiro, Brazil. The objective was to evaluate whether COMPOOTIM could use SPRL data to compose processes for three software development projects, according to the context of these projects and optimizing collaboration.

The SPRL creation effort was performed during a month, with weekly sessions among the researcher and the subject to review and discuss the created artifacts. The SPRL was created from process models and standards that describe in detail the organization's processes (Batista et al., 2013). These documents describe the activities,

roles, and artifacts involved in software development processes. In this study, the object oriented (OO) development methodology was selected among 12 methodologies that co-exist in the organization. Activities, roles, and artifacts coming from other methodologies, such as traditional project management, agile project management, testing, and software configuration management, were also included as components of the SPRL used as the basis for process composition and optimization.

The subject of the proposed experimental study was the manager of the company's software engineering process group (SEPG). His profile indicates many years of experience in software engineering, project management and software process definition. However, the participant did not have specific experience in software process reuse or composition. Regarding the organization's software processes, the subject claimed to be familiar with most of them. As responsible for and expert in software process definition, he seemed the most suitable person to participate in this study. He participated in the SPRL creation and evaluated the resulting processes from a human specialist standpoint.

Two out of the three projects used in this study were selected by the subject (with limited prior knowledge on how the experiment would be conducted) and characterized by him according to the context information adopted in the organization. Thereafter, COMPOOTIM built a process for each of these projects, following the steps described in Sections 3 and 4. The resulting processes were presented to the subject to verify if they made sense from the specialist's point-of-view. The context of the third project was simulated and also submitted to COMPOOTIM for optimization, but as this was not a real project, it was not analyzed by the subject. The third instance was much larger than the former ones and was synthesized to allow evaluating the performance of different search algorithms in the large.

Five instruments⁶ were designed and used in this study: (i) a *term of consent*, which declared the purpose of the study and ensured data confidentiality; (ii) a *subject characterization form*, to determine

⁶ These instruments are available (in Portuguese) at: <http://reuse.cos.ufrj.br/cdsoft/compootim-estudo-criacaolinha.html>.

the participants' profile; (iii) *training materials*, used to explain the main concepts underlying the usage of a SPRL and COMPOOTIM; (iv) a *project characterization form*, which allows the researcher to capture context information for each project, assisting in further data analysis; and (v) a *survey*, intended to collect qualitative information about the study and the tool.

5.2. Parameter settings

SBSE algorithms use different strategies to identify optimal or near-optimal solutions for a given problem. These strategies evaluate a number of candidate solutions, without analyzing all possible alternatives. This feature is particularly useful when the analysis of all alternatives cannot be concluded in acceptable computer processing time.

To select the algorithms under evaluation, we adopted the strategy suggested by Clarke et al. (2003): start by the simplest techniques to verify if the results are encouraging before investing in more sophisticated techniques, which may represent an additional and unnecessary effort. Therefore, this experiment used random search (RS), hill climbing (HC), and a genetic algorithm (GA) to find solutions for the PCCO problem.

Random search is a non-systematic search algorithm. It is an iterative algorithm that seeks solutions to a problem by randomly sampling and storing the best solution found so far. It starts with a randomly created candidate solution, which is evaluated by the fitness function. In each iteration, a new solution is randomly generated and evaluated by the same fitness function. If the new solution has better fitness than the current solution, it is selected as the best known solution so far. The process is repeated until a previously specified number of random solutions have been generated. This algorithm ends when the maximum number of evaluations is reached. In this experiment, we have used a budget of N^2 evaluations, where N is the number of optional components in the project under evaluation. Random search is one of the simplest search strategies available and is used as a "headless-chicken" test for more sophisticated search strategies.

Hill climbing is a local search algorithm known and applied in many SBSE problems. The algorithm starts with a randomly-crafted candidate solution (a set of process components assigned or not to the process), which is evaluated by the collaboration fitness function. The solution is stored as the best solution known so far and the main loop of the search starts. The loop attempts to find solutions with better fitness by varying the initial solution, i.e., including or removing process components (a set of neighbor solutions) in each iteration. The first process component is selected and a binary change on it is evaluated. The search follows a similar procedure to the second process component, the third, and so on. If any neighboring solution has better fitness than the current one, it is selected as the current solution and the process begins again. If no neighbor solution has better fitness than the current one, the search triggers a *random restart* in which a new, random solution (*restart solution*) is created by the same procedure that built the first, random solution. The restart solution is then used as starting point to the main loop. The limit of random restarts applied was 100. The search stops when a predefined budget of fitness evaluations is consumed and we have used the same budget provided to RS in HC.

Genetic algorithms are search algorithms based on the mechanisms of natural selection and natural genetics (Goldberg, 1989). They combine survival of the fittest, fitness-oriented reproduction, and random mutation processes to evolve a population of candidate solutions for a problem. After many generations, the evolutionary process is expected to carry the population towards being comprised of a set of individuals representing adequate solutions to the problem at hand. In the experiments performed to evaluate the adequacy of genetic algorithms to find proper solutions for the PCCO problem, we represent each individual as a string of 0s and 1s. The string has one

Table 8

Projects instances with their set of process components.

Project	Process components			
	Mandatory	Optional	Unfeasible	Total
A	25	15	2	42
B	14	8	0	22
C	25	51	3	79

entry for each optional component in the process and an entry is filled with 1 if the associated component is to be considered in the process while evaluating the solution represented by that individual. We have used a mono-objective genetic algorithm with elitism (the two better individuals of a population are carried to the next population), binary tournament (to select individuals for reproduction), single-point crossover with 80% of probability (for creating offspring based on two individuals selected for reproduction), and bit-flip mutation with 2% probability. To allow a fair comparison between different algorithms, the genetic algorithm stops when a predefined budget of fitness evaluations is consumed (the same amount used for the RS and HC algorithms).

Besides RS, HC, and GA, brute force (BF) was also implemented to be used as a benchmark for comparing the results produced by the former algorithms. BF evaluated all possible combinations of process components for a given instance, producing the set of processes with the exact maximum collaboration affordable using the available components. BF was executed for projects A and B, given that project C was too large to allow evaluating all of its potential solutions in feasible time. The algorithms were implemented using the JMetal heuristic search framework (Durillo et al., 2006). JMetal is written in Java and supports the implementation, execution, and evaluation of mono- and multi-objective heuristic search algorithms.

5.3. Problem instances

This study used three project instances. Project A followed a traditional, plan-driven software development model, was based on SAP, was developed by a team with approximately 15 members, and took almost one year to complete. Project B followed an agile software development model, was developed in Java by a team with 08 members, and had about 6 months of duration. Finally, project C was artificially created based on SPRL components considering a complex context of development. In the SPRL, the organization defined a list of 24 context information. The specific context information of projects A and B was characterized by the subject. COMPOOTIM used this context information to filter the process components that might be applicable to each project. Due to a non-disclosure agreement, project instances data cannot be made available.

The input of the optimization process was the component list filtered by COMPOOTIM for projects A, B, and C (Table 8). Project A had 42 components, being 25 mandatory, 15 optional and 2 unfeasible (components whose required artifacts would not be developed as part of the project). Project B comprised 22 components, being 14 mandatory and 8 optional. Project C included a total of 79 components, being 25 mandatory, 51 optional and 3 unfeasible.

5.4. Data collection

RS, HC, and GA were executed 30 times for each instance. Each running cycle resulted in a single best solution whose fitness value was collected. Thus, a series of 30 observations of fitness value was recorded for each instance and algorithm. Execution time was found to be very small for all algorithms: GA consumed in average 0.15 s, while HC and RS executed in about one tenth of this time for the largest instance. Therefore, we have not conducted a formal analysis

Table 9

Fitness and execution time for genetic algorithm (GA), hill climbing (HC), random search (RS) and brute force. Data indicates that HC outperforms RS for all instances, but consumes slightly more processing time than RS.

	Brute force		Hill climbing			Analysis (HC > RS)		Analysis (GA > HC)	
	Solutions	Fitness	GA	HC	RS	p-value	ES	p-value	ES
A	576	237	236.4 ± 1.1	227.1 ± 4.5	227.0 ± 4.0	0.858	51%	<0.001	99%
B	24	186	183.4 ± 3.8	184.3 ± 3.5	183.6 ± 2.6	0.046	64%	0.197	42%
C	n/a	n/a	459.0 ± 0.0	392.7 ± 7.8	384.4 ± 7.3	<0.001	81%	<0.001	100%

of execution time. All executions were performed in a computer with an eight-core i7 Intel processor running at 2600 MHz and 4GB of RAM memory. The optimization ran over Windows 7 Professional and minimal platform software required to run the Java implementation of the algorithms.

Algorithms were compared pair-wise on a per instance basis. Larger fitness values denoted that an algorithm produced more effective results than its competing one. These values were subjected to statistical inference tests to ascertain whether there was significant difference between results produced by different algorithms. Two research questions (RQ1 and RQ2, stated below) were addressed by this experiment. Section 6 presents the data analysis supporting the evaluation of these questions based on data collected after executing the experiment.

RQ1: Can a hill climbing search produce solutions with higher fitness than random search?

RQ2: Can a genetic algorithm search produce solutions with higher fitness than hill climbing?

6. Evaluation results and analysis

Table 9 summarizes the results of running GA, HC, RS, and BF searches for the selected project instances. BF was executed to evaluate if the other algorithms were able to find near-to-optimal solutions while evaluating projects A and B. Table 9 presents the mean and standard deviation of the fitness values obtained at the end of 30 execution cycles for each stochastic algorithm. Besides fitness value ranges, we applied statistical inference tests to determine whether the means for GA, HC, and RS were significantly different and calculated effect-size measures. Considering our research questions, we have first compared HC with RS (RQ1) and then compared GA with HC (RQ2).

The data sets under analysis were comprised of positive, integer numbers with an upper-bound defined by the BF algorithm. They are not normally distributed and, thus, parametric inference tests are not adequate to evaluate the statistical significance of differences between their means. Therefore, we relied on the non-parametric Wilcoxon–Mann–Whitney test for the analysis discussed below. *p*-values for the Wilcoxon–Mann–Whitney test are shown in the *p*-value columns in Table 9 for both comparisons. All values were calculated using the R Statistical Computing system v2.13.0. *p*-values close to zero indicate strong confidence that the results being compared are statistically different.

Effect-size measures, such as the non-parametric Vargha and Delaney's $\hat{A}_{12}$ statistics (Arcuri and Briand, 2011) used in ES columns in Table 9, assess the magnitude of improvement in a pair-wise comparison. Given a measure *M* for observations collected after applying treatments A and B, $\hat{A}_{12}$ measures the probability that treatment A yields higher *M* values than B. If both treatments are equivalent, then $\hat{A}_{12} = 0.5$; otherwise, it indicates the frequency of improvement. For instance, $\hat{A}_{12} = 0.7$ denotes that higher results would be obtained 70% of the time with A.

The first observation extracted from Table 9 is that project B is so small (it has only eight optional components, as presented in Table 8) that the budget of fitness function evaluations enables the

examination of all possible solutions. In fact, for such small search space, it is not necessary to apply a heuristic optimization algorithm. Whenever COMPOOTIM deals with small processes, the BF algorithm can be used, because its running time in such cases is negligible. BF was able to find 24 optimal solutions for project B with fitness value of 186. This value was a benchmark compared to results produced by GA, HC, and RS. HC achieved higher average fitness (184.3) for this instance, than RS (183.6), and finally GA (183.4). These differences are barely significant if HC is compared to RS (*p*-value < 0.05 with effect-size of 64%) and not significant at all for the comparison between HC and GA.

Regarding the larger projects, GA produced solutions with higher fitness than HC and RS for projects A and C. For project C, it was not feasible to execute the BF algorithm, but by analyzing project A results one notices that results produced by GA (236.4) are very close to the maximum fitness found by BF (237). HC comes next, with results significantly smaller than GA (*p*-values smaller than 0.001 and effect-sizes larger than 99% in favor of GA). Results produced by HC and RS are similar for instance A, but differ significantly in favor of HC for the largest instance (project C).

Summarizing, our results show that GA is more effective (best fitness values) than HC and RS in the search of solutions for collaboration optimization in process composition for large instances. However, all algorithms are competitive for small instances and even BF can be a feasible, deterministic alternative in such a context. The box-plot graph shown in Fig. 3 confirms that the GA fitness results are superior to those produced by HC and RS. In the chart, one can visually observe that GA clearly tends to find better solutions (higher fitness, presented on the Y-axis) for instances with medium to large size (projects A and C). Also, the superiority of GA increases as the instance size grows. All these findings enforce the results discussed above: RS and HC are unable to attain the same effectiveness of GA while searching for solutions to PCCO that maximize collaboration.

6.1. Specialist feedback

After running the optimization, the resulting process options were presented to the SEPG manager for evaluation. This qualitative and exploratory stage of the evaluation was performed in a single session with 1 h duration. During this session, the subject answered a survey with 15 questions, designed to record his impressions and comments about the results of the software process composition performed by COMPOOTIM based on his software process components.

As mentioned before, the SEPG manager is responsible for maintaining organizational software processes and he is an expert in software process definition. Therefore, he is in contact with many software development projects carried on by the organization and was able to characterize projects A and B contexts. Considering the subject's experience, he can judge the feasibility of the practical use of process composition in the organization.

As shown in Table 9, a total of 576 distinct feasible process compositions were found for project A, while 24 feasible compositions were found for project B. These process composition alternatives vary in relation to the collaboration potential achieved, the number and order of process components to be executed, and the number of artifacts

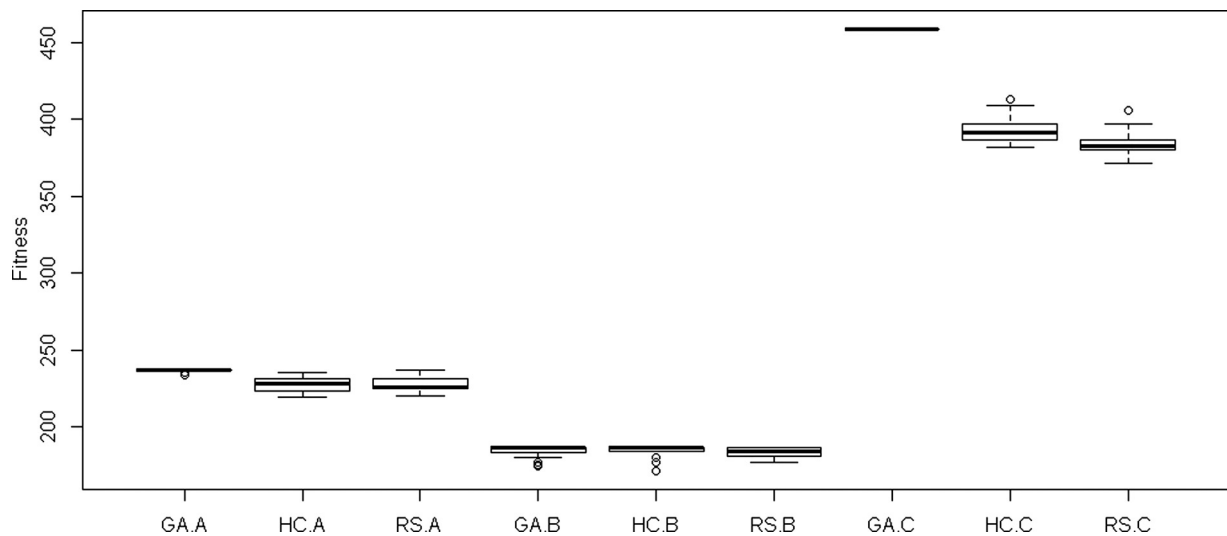


Fig. 3. Box-plot of fitness values produced by different algorithms (GA, HC, and RS) for projects A, B and C. We can observe that GA produces higher fitness values than its competing algorithms, especially for the largest project.

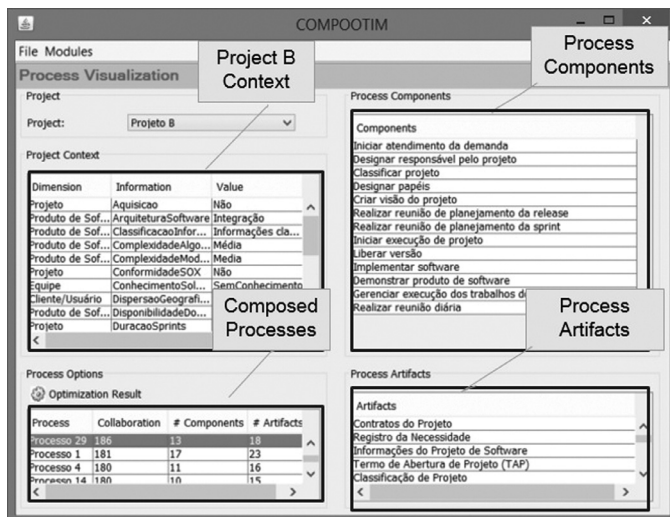


Fig. 4. Optimization results presented on COMPOOTIM.

produced by the process. The context of the third project was simulated and also submitted to COMPOOTIM for optimization, but it was not analyzed by the subject.

Using COMPOOTIM's dashboard (exemplified in Fig. 4), the subject could browse a selection of the 10 best process compositions with the following information for each one to evaluate the differences among them: collaboration potential, total number of components, total number of artifacts, list of components and list of artifacts. However, the specialist was not previously informed that we consider these the best alternatives.

Through the applied survey, the specialist indicated a high level of satisfaction with the resulting processes. About the size and completeness of the processes suggested for projects A and B, the subject reported that they were in accordance with the original organizational SPRL and matched his expectation considering the variations between traditional and agile development methods adopted. In addition, he highlighted that having feasible process compositions of different sizes (in relation to the number of process components or produced artifacts) was an interesting contribution.

The main advantage identified by the specialist was the "ability to identify the best process scenario to a specific development project.

It can help project managers who do not have much knowledge about software engineering". This observation is an important indication that COMPOOTIM achieved its objective of supporting a project manager during decision making in process composition.

On the other hand, the main challenge mentioned was the need to improve the presentation of results in the tool (Fig. 4). Indeed, in the current dashboard the possibilities of parallelism in the process are not clear. Thus, this visualization could be evolved in future work. Besides collaboration, the manager of SEPG suggested improving the ability to consider "other factors that could help in process composition, e.g., lessons learned and results of the projects". Such a suggestion points to the opportunity to optimize factors which were not addressed in the current research in process composition in the future.

6.2. Threats to validity

This section discusses the main limitations of the proposed experimental setting, including the threats to validity. Wohlin et al. (1999) classify the threats to validity that may affect an experimental study into four categories: conclusion, construct, internal, and external threats.

The main limitation of this study is the difficulty to generalize the results because of the restricted sample of instances in terms of company, subject, and projects. First, the experiment used SPRL data from a single company. However, we must consider that this is one important oil company in Brazil with national geographic distribution. In addition, actual organization software processes based on different software methodologies were used to create the SPRL. This SPRL can be used in the future both by the organization to the definition of internal software processes as by our researches in other experimental studies.

The small number of projects used to evaluate the performance of heuristic algorithms is another concern. Although these projects have different characteristics and context information, it is not possible to generalize the results to other types of projects or projects in different contexts. Performing an analysis with a larger number of projects would be important to draw definitive conclusions about the suitability of these algorithms for the PCCO problem or the need to implement more sophisticated algorithms. Nevertheless, finding real and interesting instances to be solved is not simple. Processes tend to be strategic for most companies and in many cases they do not allow a research to collect the required data. Besides, this data tends to lack a proper structure and to require significant effort to be collected and organized.

Conclusion threats are concerned with the relationship between treatment and outcome. In this regard, conclusions in SBSE experimental studies may not consider random variation in the search, poorly summarize the data, or lack of a meaningful comparison baseline (Barros and Dias-Neto, 2011). In the present research, these issues were addressed by: (i) running 30 cycles for each instance; (ii) presenting central and dispersion measures (mean and standard deviation) as well as *p*-values and effect sizes; (iii) determining the adherence of data to a particular theoretical distribution and applying an appropriate statistical inference test; and (iv) by comparing the heuristic algorithms (HC and RS) with a brute force algorithm serving as benchmark for small instances.

In addition, one should notice that no control was applied in this experiment. If the process options that were not in the rank of better alternatives were also presented to the subject to evaluation, it should be possible to compare the results. However, this initial study, with humans, aims to complement the analysis of more algorithmic nature. Such kind of improvement can be adopted in future replications of this study, where a selection of solutions of varying quality is presented to the subjects to rank them in terms of overall quality and the level of collaboration they are likely to engender.

Finally, as the measurement of collaboration is still an open research topic in the literature (Thomson et al., 2009), there is lack of empirical evidence to support the proposed fitness function.

Internal threats evaluate if the relationship between treatment and outcome is causal or results from factors which the researcher cannot control. Major internal threats in SBSE experiments involve poor parameterization, lack of real-world problem instances, lack of a detailed description about code instrumentation and data collection procedures. In this paper we have used two real-world instances of distinct sizes and a large synthetic instance. We have also described the data collection procedure, but the source-code and instances cannot be made available due to a non-disclosure agreement with the company that provided the data for our experimental analysis.

Construct threats are concerned with the relation between theory and observation, ensuring that the treatment reflects the construct of the cause and that the outcome reflects the construct of the effect. Major construct threats in SBSE experiments involve using unreliable efficiency and effectiveness measures and not discussing the underlying model subjected to optimization. On regards to these issues, we have presented a detailed problem definition in Section 4.1 and have used the collaboration measurement strategy discussed in Section 4.2. Finally, the use of wall-clock time as a measure for efficiency is questionable, but all experiments were executed in similar computers with compatible load.

Finally, *external threats* are concerned with the generalization of the observed results to a larger population, outside the sample instances used in the experiment. Major external threats to SBSE experiments include the lack of a clear definition of target instances, unclear instance selection strategy, and not having enough diversity in instance size and complexity. We have used three instances of different sizes (in terms of number of process components) and complexity (in terms of number of provided and required artifacts). These instances and their selection were described in Section 5.3. Unfortunately, they are not available for download and further evaluation due to confidentiality constraints.

7. Related work

The work of Martínez-Ruiz et al. (2011) is closely related to this work, because it applies rationale management to support decision making when tailoring processes. However, our work differentiates by adopting a strategy of processes composition and reuse, instead of tailoring. In addition, the use of rationale management can help to share the knowledge about process definition, but the authors apply brute force and do not propose mechanisms to automate it.

Two closely related areas are contingency factors (CFs) and situational method engineering (SME). CFs (Bekkers et al., 2008) refer to variables that characterize a project to determine the selection of an appropriate method from a portfolio. This idea of project characterization is similar to the one proposed in this work of context information use. However, this idea was expanded: as part of a context management approach, it offers a greater dynamism in the selection of processes and also enables the consideration of the team and organization context. Furthermore, there is a lack of guidelines on how CFs relate to the development methods (Kumar and Welke, 1992). This limitation is overcome in our work through the use of context rules that relate context definitions to process components.

The area of SME (Brinkkemper, 1996) is intended to provide organizations with flexibility in configuring a project-specific process, using methods or fragments stored in a repository. SME also proposes facilities for this process configuration. This support and the existence of a range of process knowledge are similarities that approximate the COMPOOTIM approach to SME. However, one fundamental problem is that project managers need assistance to make decisions about the process to be adopted in the project. While SME can present all the details about each individual process fragment, it gives no indication about the selection and combination of them to guarantee that the resulting process is usable. The capacity to compose this process, considering the project context, is a contribution of COMPOOTIM.

SME and CFs can be combined to select methods according to the project context. In our work, the idea is not to identify one method that can be used in its entirety, but to compose a process using components from a process line that has been created based on different development models.

Despite the existence of proposals that deal with parts of this solution, like process lines (Aleixo et al., 2010; Barreto et al., 2010; Rombach, 2006; Washizaki, 2006), they do not tackle the problem with a complete, integrated, and dynamic solution as in this work.

Regarding optimization, Karlstrom and Runeson (2002) propose an optimization approach for the introduction of extreme programming (XP) in organizations. This proposal suggests practices that should be selected, considering the ease of adoption and the potential beneficial effects of each practice. Again, the selection of one individual practice is not the focus of our work but to present a solution for a software process composition.

8. Conclusions

This paper presented the COMPOOTIM approach to collaboration management – planning, composing, optimizing, and monitoring – in software processes. This approach was evaluated through an experimental study conducted in the context of a large oil and gas company in Brazil using three projects. The objective was to evaluate the feasibility of maximizing collaboration while composing software processes. The motivation of the company in providing a professional at management level to participate in this study shows that it is an actual problem in the software industry.

The first part of this study compared the performance of a heuristic search technique – GA and HC with RS in optimizing collaboration in process composition. The aim was to determine which search algorithm would be more effective in addressing the problem. BF was also implemented to be used as a benchmark for comparing results produced by the algorithms for small instances. As a result, it was observed that regarding the larger projects, GA produced solutions with higher fitness than HC and RS. For small search space, it is not necessary to apply a heuristic optimization algorithm – BF can be used, because its running time in such cases is negligible.

The second part of the study was human-oriented. A specialist point-of-view was explored to check whether the resulting process produced by search-based optimization was reasonable in terms of size, complexity, diversity, and sequence of components. The

specialist indicated that he was satisfied with the resulting processes. He highlighted that having software processes of different sizes (in relation to the number of process components or produced artifacts) was an interesting contribution. Finally, he observed that COMPOOTIM might be even more important to support even less experienced project managers during decision making on regards to process composition.

8.1. Future work

Our research agenda also involves investigating the **execution** of the composed software processes. We intend to verify if the planned collaboration level is really achieved in practice when the optimized processes are executed as part of the daily business of the software development branch of a company. Then, we intend to investigate process adaptation on-the-fly, i.e., changing the process during execution while keeping collaboration as close as possible to a maximum. Dynamic (Cinnéide et al., 2012) and incremental (Barros, 2013) SBSE techniques are expected to be useful here.

On regards to **collaboration**, we are focused on improving our proposed measurement strategy to obtain refined ways to determine the collaboration potential. Starting from the proposed measurement strategy, it is possible to evolve the research assigning different weights to determine to which extent a particular component positively or negatively contributes to a particular collaboration practice.

In theory, a process component has potential to contribute to several collaboration practices, but during its execution this may not happen. The measurement strategy, in fact, is an indication assigned during the collaboration planning. However, this estimation may not be confirmed during process execution. So, while monitoring collaboration, there is a need to assess the collaboration potential values assumed for each component at running time and, possibly, change plans for the software process according to reality.

Regarding **optimization** mechanisms implemented in COMPOOTIM, the fact that the algorithms have been evaluated based on a limited number of projects can be considered a limitation. The use of a larger number of projects would be important to enforce conclusions about the suitability of the selected algorithm (GA and HC) to find solutions for the PCCO problem or the need for more sophisticated algorithms, such as multi-objective algorithms.

The interest of the company where the study was conducted in maintaining a joint research represents a concrete opportunity for future expansion of the SPRL and to conduct further experimental **evaluations**. Thus, to complement the assessments conducted in this work, the experimental study could be repeated with a larger amount of projects in order to reduce threats to validity. It can also be interesting to see how the specialist would rank the process options, during the evaluation of both the process options obtained from the COMPOOTIM and process options created by a project manager for the same project.

Sensitivity analysis is another future work in potential. It enables the investigation of the degree of influence that each of the input parameters cause in the solution and identifying which parameters (number of cycles, limit of budget evaluations, etc.) contribute most to the location of the optimal solution. This analysis consists of selecting each of the parameters, varying its value while the values of others are fixed and observing the effects on the quality of the solution generated and response time.

Finally, it should be noticed that there are many possible aspects of interest that can be considered as optimization objects, in addition to collaboration, in the context of COMPOOTIM. For instance, the need for discipline/control/formalism can work as a counterpoint to collaboration and be balanced with the latter in software process composition (Magdaleno, 2010b). It would avoid that collaboration was optimized as a single and isolated factor and without any kind of cost ver-

sus benefit analysis. This opportunity also opens space for an expansion of the use of optimization algorithms, since bi-objectives search algorithms might find interesting combinations of both aspects while evaluating different combinations of software process components.

Acknowledgment

This research was partially funded by CNPq (grant numbers 151158/2013-4, 300678/2012-6) and FAPERJ (grant numbers E-26/102.285/2013).

References

- Alba, E., Chicano, J.F., 2007. Software project management with GAs. *Inf. Sci.* 177 (11), 2380–2401.
- Aleixo, F.A., Freire, M.A., Santos, W.C., et al., 2010. A model-driven approach to managing and customizing software process variabilities. In: *International Conference on Enterprise Information Systems (ICEIS)*. Funchal, Madeira, Portugal, pp. 92–100.
- Alonso, J., de Soria, I.M., Orue-Echevarria, L., 2010. Enterprise Collaboration Maturity Model (ECMM): preliminary definition and future challenges, *Enterprise Interoperability IV*. Springer, London, pp. 429–438.
- Altmann, J., Pomberger, G., 1999. Cooperative software development: concepts, model and tools, *Technology of Object-Oriented Languages and Systems (TOOLS)*. Santa Barbara, CA, USA, pp. 194–207.
- Antoniol, G., Di Penta, M., Harman, M., 2004. A robust search-based approach to project management in the presence of abandonment, rework, error and uncertainty. In: *Proceedings of the 10th International Symposium on Software Metrics (METRICS)*. Chicago, IL, USA, pp. 172–183.
- Araujo, R.M., de, Borges, M.R.S., 2007. The role of collaborative support to promote participation and commitment in software development teams. *Softw. Process Improv. Pract.* 12 (3), 229–246.
- Arcuri, A., Briand, L., 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In: *Proceedings of the 33rd International Conference on Software Engineering*. New York, NY, USA, pp. 1–10.
- Armbrust, O., Katahira, M., Miyamoto, Y., et al., 2008. Scoping software process models – initial concepts and experience from defining space standards, *Making Globally Distributed Software Development a Success Story*. Springer, Berlin/Heidelberg, pp. 160–172.
- Bagnall, A.J., Rayward-Smith, V.J., Whitley, L.M., 2001. The next release problem. *Inf. Softw. Technol.* 43 (14), 883–890.
- Barreto, A., Barros, M. de O., Werner, C.M.L., 2008. Staffing a software project: a constraint satisfaction and optimization-based approach. *Comput. Oper. Res.* 35 (10), 3073–3089.
- Barreto, A., Rocha, A.R., Murta, L., 2010. Supporting the definition of software processes at consulting organizations via software process lines. In: *International Conference on the Quality of Information and Communications Technology (QUATIC)*. Porto, Portugal, pp. 15–24.
- Barreto, A.S., Murta, L.G.P., Cavalcanti da Rocha, A.R., 2011. Software process definition: a reuse-based approach. *J. Universal Comput. Sci.* 17 (13), 1765–1799.
- Barros, M. de O., 2012. An analysis of the effects of composite objectives in multiobjective software module clustering. In: *Proceedings of the Fourteenth International Conference on Genetic and Evolutionary Computation Conference*. New York, NY, USA, pp. 1205–1212.
- Barros, M. de O., 2013. An experimental study on incremental search-based software engineering. In: Ruhe, G., Zhang, Y. (Eds.), *Search Based Software Engineering*. Springer, Berlin Heidelberg, pp. 34–49.
- Barros, M.O., Dias-Neto, A.C., 2011. Threats to Validity in Search-based Software Engineering Empirical Studies, *Relatório Técnico 0006/2011, RelatTe-DIA, UNIRIO*.
- Batista, C.F.A., Magdaleno, A.M., Werner, C.M.L., et al., 2013. A Experiência da Petrobras na Criação de uma Linha de Processos de Software. *Congresso Brasileiro de Software: Teoria e Prática (CBSOFT) – Trilha da Indústria*.
- Beck, K., Beedle, M., Bennekum, A. van, et al., 2001. Manifesto for Agile Software Development. Available at: <http://agilemanifesto.org/>. Last access: 15 December 2008.
- Bekkers, W., van de Weerd, I., Brinkkemper, S., et al., 2008. The influence of situational factors in software product management: an empirical study, *International Workshop on Software Product Management (IWSPM)*, Barcelona, Catalonia, Spain, pp. 41–48.
- Borrelli, G., Cable, J., Higgs, M., 1995. What makes teams work better. *Team Perform. Manage.* 1 (3), 28–34.
- Bosch, J., Bosch-Sijtsema, P.M., et al., 2010. Softwares product lines, global development and ecosystems: collaboration in software engineering. In: Mistrik, I., Grundy, J., Hoek, A. (Eds.), *Collaborative Software Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 77–92.
- Brinkkemper, S., 1996. Method engineering: engineering of information systems development methods and tools. *Inf. Softw. Technol.* 38 (4), 275–280.
- Cataldo, M., Herbsleb, J.D., 2008. Communication networks in geographically distributed software development. In: *Proceedings of the 2008 ACM conference on Computer supported cooperative work*. San Diego, CA, USA, pp. 579–588.
- Chesbrough, H.W., 2003. The era of open innovation. *MIT Sloan Manage. Rev.* 44 (3), 35–41.
- Chrissis, M.B., Konrad, M., Shrum, S., 2006. *CMMI: Guidelines for Process Integration and Product Improvement*, 2nd ed. Addison-Wesley, Boston, USA.

- Cinnéide, M., Tratt, L., Harman, M., et al., 2012. Experimental assessment of software metrics using automated refactoring. In: *Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*. New York, NY, USA, pp. 49–58.
- Clarke, J., Dolado, J.J., Harman, M., et al., 2003. Reformulating software engineering as a search problem. *IEEE Proc. Softw.* 150 (3), 161–175.
- Cugola, G., Ghezzi, C., 1998. Software processes: a retrospective and a path to the future. *Softw. Process Improv. Pract. J.* 4 (3), 101–123.
- Dean, D.L., Deokar, A., Ter Bush, R., 2006. Making the collaboration engineering investment decision. In: *Hawaii International Conference on System Sciences (HICSS)*. Kauai, Hawaii, USA, pp. 1–10.
- Deelstra, S., Sinnema, M., Bosch, J., 2005. Product derivation in software product families: a case study. *J. Syst. Softw.* 74 (2), 173–194.
- DeMarco, T., Lister, T., 1999. *Peopleware: Productive Projects and Teams*, 2nd ed. New York, USA.
- Dourish, P., Bellotti, V., 1992. Awareness and coordination in shared workspaces. In: *Conference on Computer Supported Cooperative Work (CSCW)*. Toronto, Ontario, Canada, pp. 107–114.
- Durillo, J.J., Nebro, A.J., Luna, F., et al., 2006. JMetal: A Java Framework for Developing Multi-objective Optimization Metaheuristics.
- Ellis, C.A., Gibbs, S.J., Rein, G., 1991. Groupware: some issues and experiences. *Commun. ACM* 34 (1), 39–58.
- Feiler, P., Humphrey, W.S., 1992. *Software Process Development and Enactment: Concepts and Definitions*.
- Fischer, L., 2011. Social BPM: Work, Planning and Collaboration under the Impact of Social Technology.
- Fuggetta, A., 2000. Software process: a roadmap. In: *Proceedings of the Conference on the Future of Software Engineering*. Limerick, Ireland, pp. 25–34.
- Gary, K.A., Lindquist, T.E., 1999. Cooperating process components. In: *International Computer Software and Applications Conference (COMPSAC)*. Phoenix, AZ, USA, pp. 218–223.
- Gerosa, M., Pimentel, M., Fuks, H., et al., 2006. Development of groupware based on the 3C collaboration model and component technology, *Groupware: Design, Implementation, and Use*, pp. 302–309.
- Goldberg, D.E., 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*, 1st ed.
- Hansen, M.T., 2009. When internal collaboration is bad for your company. *Harv. Bus. Rev.* 84 (3), 83–88.
- Harman, M., 2007. The current state and future of search based software engineering. In: *2007 Future of Software Engineering*, pp. 342–357.
- Harman, M., Jones, B.F., 2001. Search-based software engineering. *Inf. Softw. Technol.* 43, 833–839.
- Harman, M., Mansouri, S.A., Zhang, Y., 2009. Search based software engineering: a comprehensive analysis and review of trends techniques and applications, Technical Report.
- Herbsleb, J.D., Paulish, D.J., Bass, M., 2005. Global software development at Siemens: experience from nine projects. In: *Proceedings of the 27th International Conference on Software Engineering*. St. Louis, MO, USA, pp. 524–533.
- Hollenbach, C., Frakes, W., 1996. Software process reuse in an industrial setting. In: *International Conference on Software Reuse (ICSR)*, pp. 22–30.
- Karlström, D., Runeson, P., 2002. Decision support for extreme programming introduction and practice selection. In: *Proceedings of the 14th International Conference on Software Engineering and Knowledge Engineering*. Ischia, Italy, pp. 835–841.
- Khoshafian, S., Buckiewicz, M., 1995. *Introduction to Groupware, Workflow, and Workgroup Computing*. Wiley.
- Khoshgoftar, T.M., Liu, Y., Seliya, N., 2004. A multiobjective module-order model for software quality enhancement. *IEEE Trans. Evol. Comput.* 8 (6), 593–608.
- Kumar, K., Welke, R.J., 1992. *Methodology Engineering: a proposal for situation-specific methodology construction, Challenges and Strategies for Research in Systems Development*. John Wiley & Sons, pp. 257–269.
- Lee, K., Kang, K.C., Lee, J., 2002. Concepts and guidelines of feature modeling for product line software engineering. In: *Proceedings of the 7th International Conference on Software Reuse: Methods, Techniques, and Tools*, pp. 62–77.
- Li, Z., Harman, M., Hierons, R.M., 2007. Search algorithms for regression test case prioritization. *IEEE Trans. Softw. Eng.* 33 (4), 225–237.
- Lindvall, M., Rus, I., 2000. Process diversity in software development. *Softw. IEEE* 17 (4), 14–18.
- Magdaleno, A.M., 2010a. An optimization-based approach to software development process tailoring, PhD Track – *International Symposium on Search Based Software Engineering (SSBSE)*. Benevento, Italy, pp. 40–43.
- Magdaleno, A.M., 2010b. Balancing collaboration and discipline in software development processes, *Doctoral Symposium of International Conference on Software Engineering (ICSE)*. Cape Town, South Africa, pp. 331–332.
- Magdaleno, A.M., Araujo, R.M., Borges, M.R.S., 2009. A maturity model to promote collaboration in business processes. *Int. J. Bus. Process Integr. Manage.* 4 (2), 111–123.
- Magdaleno, A.M., Araujo, R.M., Werner, C.M.L., 2012. COMPOOTIM: An approach to software processes composition and optimization, *Congresso Ibero-Americano em Engenharia de Software (CIBSE)*. Buenos Aires, Argentina, pp. 1–14.
- Magdaleno, A.M., Cappelli, C., Baiao, F.A., et al., 2008. Towards collaboration maturity in business processes: an exploratory study in oil production processes. *Inf. Syst. Manage.* 25 (4), 302–318.
- Magdaleno, A.M., Werner, C.M.L., Araujo, R.M., 2012. Reconciling software development models: a quasi-systematic review. *J. Syst. Softw.* 85 (2), 351–369.
- Magdaleno, A., Nunes, V., Araujo, R., et al., 2007. Flexible organizational process deployment, *Computer Supported Cooperative Work in Design III*. Springer, Berlin/Heidelberg, pp. 679–688.
- Marchetto, A., Francescomarino, C.D., Tonella, P., 2011. Optimizing the trade-off between complexity and conformance in process reduction. In: Cohen, M.B., Cinnéide, M.O. (Eds.), *Search Based Software Engineering*. Springer, Berlin Heidelberg, pp. 158–172.
- Martínez-Ruiz, T., García, F., Piattini, M., 2011. Managing process diversity by applying rationale management in variant rich processes, *Product-Focused Software Process Improvement*. Springer, pp. 128–142.
- Mistrik, I., Grundy, J., Hoek, A., et al., 2010. *Collaborative Software Engineering*. Berlin, Heidelberg, Springer.
- Nunes, V.T., Werner, C.M.L., Santoro, F.M., 2011. Dynamic process adaptation: a context-aware approach. In: *15th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, pp. 97–104.
- Nunes, V.T., Werner, C., Santoro, F.M., 2010. Context-based process line. In: *International Conference on Enterprise Information Systems (ICEIS)*. Funchal, Madeira, Portugal, pp. 277–282.
- OMG, 2008. *Software Process Engineering Metamodel*. Available at: <http://www.omg.org/technology/documents/formal/spem.htm>. Last access: 23 August 2008..
- Park, S., Na, H., Park, S., et al., 2006. A semi-automated filtering technique for software process tailoring using neural network. *Expert Syst. Appl.* 30 (2), 179–189.
- Pedreira, O., Piattini, M., Luaces, M.R., et al., 2007. A systematic review of software process tailoring. *SIGSOFT Softw. Eng. Notes* 32 (3), 1–6.
- Praditwong, K., Harman, M., Yao, X., 2011. Software module clustering as a multi-objective search problem. *IEEE Trans. Softw. Eng.* 37 (2), 264–282.
- Pressman, R.S., 2001. *Software Engineering: A Practitioner's Approach*, 5th ed. McGraw-Hill.
- Raman, S., 2000. It is software process, stupid: next millennium software quality key. *Aerosp. Electron. Syst. Mag. IEEE* 15 (6), 33–37.
- Raymond, E.S., 2001. *The Cathedral & the Bazaar*.
- Ribó, J.M., Franch, X., 2002. Supporting Process Reuse in PROMENADE, Research Report LSI-02-14-R, Departament de Llenguatges i Sistemes Informàtics, Universitat Politècnica de Catalunya..
- Rombach, D., 2006. *Integrated Software Process and Product Lines, Unifying the Software Process Spectrum*. Springer-Verlag, Heidelberg, pp. 83–90.
- Schönthaler, F., Vossen, G., Oberweis, A., et al., 2012. *Business Processes for Business Communities – Modeling Languages, Methods, Tools*. Springer, Berlin Heidelberg.
- Siebel, N.T., Cook, S., Satpathy, M., et al., 2003. Latitudinal and longitudinal process diversity. *J. Softw. Mainten. Res. Pract.* 15 (1), 9–25.
- Ternite, T., 2009. Process lines: a product line approach designed for process model development. In: *35th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Patras, Greece, pp. 173–180.
- Thomson, A.M., Perry, J.L., Miller, T.K., 2009. Conceptualizing and measuring collaboration. *J. Public Adm. Res. Theory* 19 (1), 23–56.
- Tomasi, A., Marchetto, A., Francescomarino, C.D., 2012. Domain-driven reduction optimization of recovered business processes. In: Fraser, G., Souza, J.T. de (Eds.), *Search Based Software Engineering*. Springer, Berlin Heidelberg, pp. 228–243.
- Vreede, G.-J., Briggs, R.O., 2005. Collaboration engineering: designing repeatable processes for high-value collaborative tasks. In: *Hawaii International Conference on System Sciences (HICSS)*. Hilton Waikoloa Village, Hawaii, USA, pp. 1–10.
- Washizaki, H., 2006. Building software process line architectures from bottom up, *Product-Focused Software Process Improvement (PROFES)*. LNCS, Amsterdam, The Netherlands, pp. 415–421.
- Whitehead, J., Mistrik, I., Grundy, J., et al., 2010. Collaborative software engineering: concepts and techniques. *Collab. Softw. Eng.* 1, 1–30.
- Wohlin, C., Runeson, P., Höst, M., 1999. *Experimentation in Software Engineering: An Introduction*, 1st ed. Springer.
- Xu, P., Ramesh, B., 2008. Using process tailoring to manage software development challenges. *IT Professional* 10 (4), 39–45.
- Yildirim, N., 2006. Interactions of software technology with technologies and social issues: interfaces and requirements for interdisciplinary collaboration, *Technology Management for the Global Future (PICMET)*. Istanbul, Turkey, pp. 356–367.
- Yoo, S., Harman, M., 2012. Regression testing minimization, selection and prioritization: a survey. *Softw. Test. Verif. Reliab.* 22 (2), 67–120.
- Zhang, Y., Harman, M., Mansouri, S.A., 2007. The multi-objective next release problem. In: *Proceedings of the 9th annual conference on Genetic and evolutionary computation*. New York, NY, USA, pp. 1129–1137.

Andréa Magalhães Magdaleno is a post-doc student at the Computer Science Department of COPPE/UFRJ, Brazil. She holds a D.Sc. in Computer Science from COPPE/UFRJ (2013). She obtained her M.Sc. degree in Computer Science at UFRJ in 2007. She is the author of publications in Brazilian and international conferences and journals. She coordinates consulting projects at companies in the field of Business Process Modeling and Requirements Engineering. Her areas of interest are: Collaboration and Business Process Management (BPM).

Márcio de Oliveira Barros is an associate professor at the Applied Informatics Department of the Federal University of Rio de Janeiro State (UNIRIO). He holds a Doctor degree in System Engineering and Computer Science from COPPE/UFRJ. His research concentrates on using optimization and simulation to describe and find (close to) optimal solutions to software engineering problems. He also uses these tools to bring forth insight about Software Engineering problems. His main research interests within Software Engineering include software design and project management.

Cláudia Maria Lima Werner holds a D.Sc. in Computer Science from COPPE/UFRJ (1992). She is an Associate Professor at COPPE/UFRJ and leads the Software Engineering group. She is also a CNPq and FAPERJ researcher. She has published several papers and served on program committees of both Brazilian and international conferences and journals. She has extensive experience in Computer Science with an emphasis on Software Engineering in the following research areas: software reuse, component-based development, and software product line.

Renata Mendes de Araujo obtained her D.Sc. in Computer Science from COPPE/UFRJ (2000). She is an Associate Professor at UNIRIO (Rio de Janeiro State Federal Univer-

sity), Brazil. Her experience and research work focus on Information Systems, Software Development Process Improvement, Business Process Management, Computer-Supported Cooperative Work and e-Democracy. Her current research concentrates on how to turn organizations more democratic by the use of business process management, collaborative support and social tools.

Carlos Freud Alves Batista is Manager of Software Processes and Methodologies for TIC/CPSW/PGOD/PMSW in Petrobras. Also, holds a M.Sc. in Computer Science from PUC-Rio (2007). He has experience in the areas of Software Engineering, Software Process Definition and Project Management.